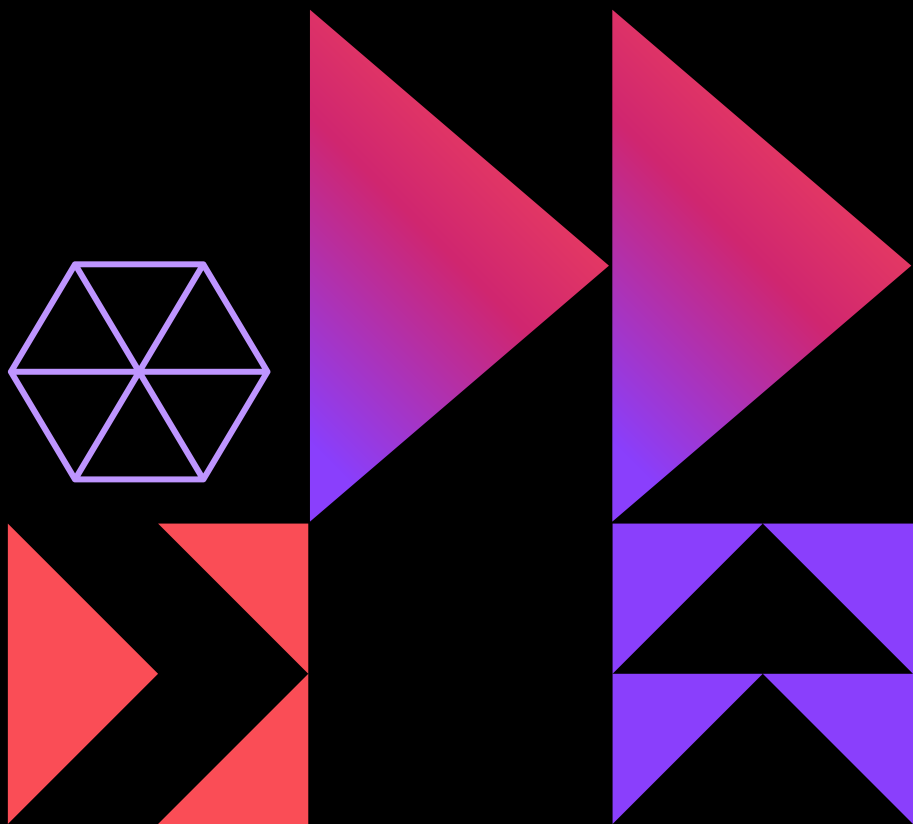
LAB-3834: Agentic RAG with OpenRAG on IBM watsonx.data & External Agents: From Docling Ingestion to MCP

Hands-on lab guide

Jens Philipp Müller

jens.mueller@de.ibm.com

IBM Expert Developer,
watsonx.data Development
– DataLabs



Version: 2026-10-22

The latest version of this document is available at:

https://ibm.biz/lab-3834-lab-guide

Table of Contents

1. Introduction 6
2. Getting Started 8
2.1. Downloading the ZIP archive required for the lab 8
3. Task: Accessing OpenRAG 9
3.1. Logging in to IBM Cloud 9
3.2. Creating an IBM Cloud API key 9
3.3. Navigating to the watsonx.data web console 10
3.4. Navigating to the OpenRAG console 10
4. Task: Ingesting files 12
5. Task: Querying documents 16
5.1. Searching documents 16
5.2. Chatting with the agent 17
5.3. Ingesting web content during a chat session 18
6. Task: Filters and querying filtered documents 19
6.1. Creating a connection to IBM Cloud Object Storage 19
6.2. Ingesting files from IBM Cloud Object Storage 20
6.3. Creating and using knowledge filters 22
7. Task: Langflow flows for ingesting and querying 24
7.1. OpenSearch Ingestion Flow 24
7.2. OpenRAG OpenSearch Agent Flow 25
7.3. Agent Instructions 27
8. Task: Connecting watsonx Orchestrate with OpenRAG via MCP 28
8.1. Navigating to watsonx Orchestrate 28
8.2. Creating a connection to the OpenRAG MCP server 28
8.3. Creating an agent 29

Notices and disclaimers

© 2025 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed "as is" without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts. In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply.

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled, isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at: www.ibm.com/legal/copytrade.shtml.

1. Introduction

AI agents and applications are only as useful as the knowledge they can access, interpret, and apply. In enterprise environments, that knowledge is often scattered across documents, formats, and systems.

OpenRAG on watsonx.data helps make enterprise content available to AI applications. It can connect to existing content sources such as Google Drive, Microsoft OneDrive, SharePoint, AWS S3, and IBM Cloud Object Storage, and work with common file types including PDFs, images, PowerPoint presentations, and Excel files. OpenRAG prepares and indexes this content, making it searchable and accessible for AI agents.

OpenRAG provides an integrated platform for the key components of Retrieval-Augmented Generation (RAG), including ingestion, indexing, retrieval, and orchestration. This allows teams to build knowledge-grounded AI applications without assembling and operating the underlying components individually.

Typical use cases include:

- Internal knowledge assistants that search enterprise documents and business content.
- Customer support assistants that retrieve product documentation and knowledge articles.
- Sales assistants that combine information from multiple business systems.
- Domain-specific agents that work with policies, procedures, records, and other enterprise content.

From Traditional RAG to Agentic Retrieval

The goal of RAG is to provide AI models with access to information beyond their training data. In a traditional RAG workflow, documents are ingested, divided into chunks, indexed, and searched when a user asks a question. Relevant content is retrieved and provided to a language model to generate an answer.

While effective for many scenarios, this approach depends heavily on a single retrieval step. Complex enterprise questions often require searching multiple sources, refining queries, combining evidence, and deciding whether additional information is needed.

OpenRAG supports an **agentic RAG** approach. Instead of relying on a fixed retrieve-and-generate pattern, an AI agent can interpret the task, perform multiple retrieval steps, refine searches, use tools, and validate results before generating a response. This enables more flexible and reliable knowledge-grounded AI applications.

OpenRAG Architecture

OpenRAG on watsonx.data is based on IBM's open-source OpenRAG framework and combines three key open-source technologies:

- **Docling** for document ingestion and preparation
- **OpenSearch** for hybrid search and retrieval
- **Langflow** for agent orchestration

These components provide an end-to-end workflow that prepares content for AI, indexes it for retrieval, and enables the creation of agentic workflows through a visual orchestration environment.

What You Will Do in This Lab

In this hands-on lab, you will explore the OpenRAG platform from end to end. You will:

1. Ingest and process documents using Docling.
2. Create and query a searchable knowledge base.
3. Explore the retrieval and orchestration pipeline.

4. Understand how agentic RAG workflows are implemented in OpenRAG.
5. Access OpenRAG through Model Context Protocol (MCP) integrations from external agents such as IBM watsonx Orchestrate.

By the end of the lab, you will have hands-on experience building, customizing, and consuming agentic RAG workflows with OpenRAG on watsonx.data.

2. Getting Started

Figure 1 illustrates the lab environment. All tasks will be carried out from the lab host, which serves as your central access point. From this host, you can utilize a web browser to complete your lab activities.

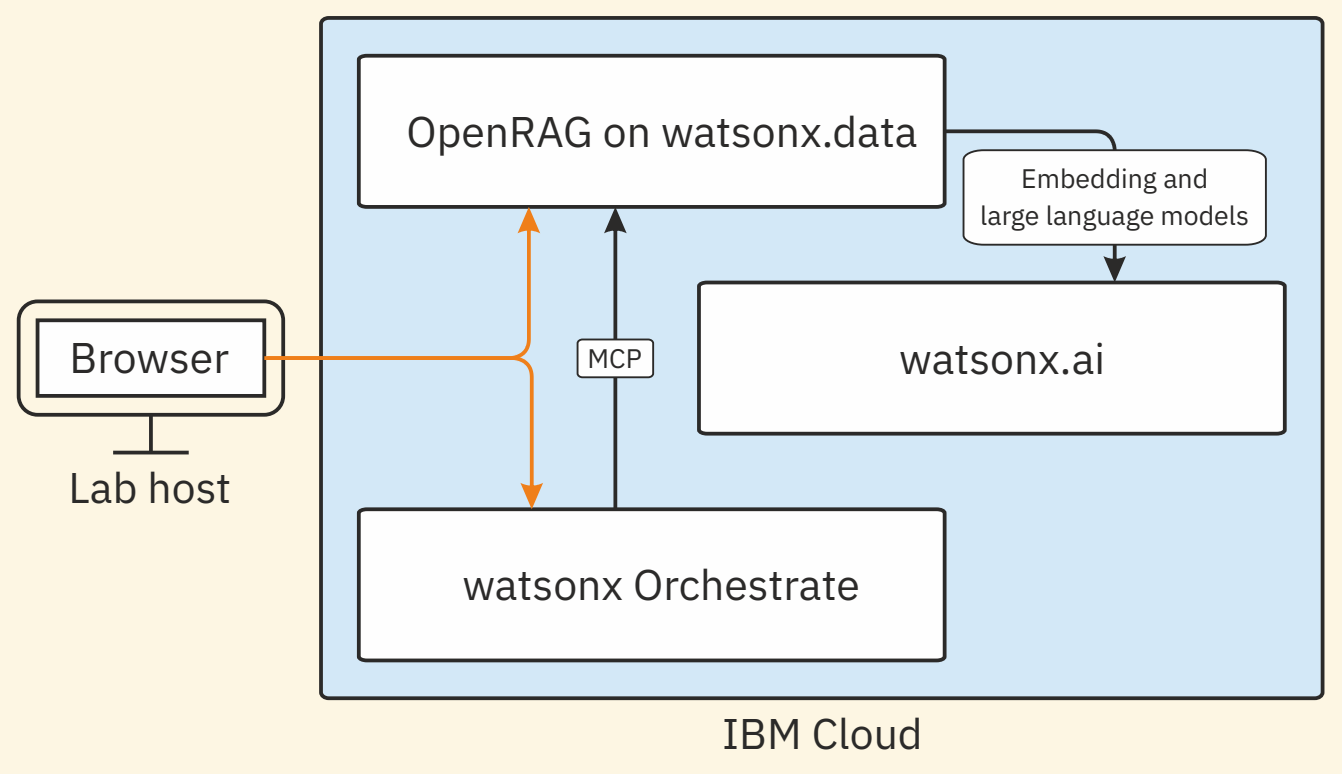


Figure 1: Lab environment

The IBM Cloud account used for this lab is shared among all participants and provides access to the following primary resources:

1. watsonx.data, providing the OpenRAG service
2. watsonx Orchestrate

2.1. Downloading the ZIP archive required for the lab

1. In the browser on your lab host, download the following ZIP archive required for the lab:

   https://ibm.biz/lab-3834

2. Extract the archive, which contains a subfolder named “OpenRAG documentation”.

3. Task: Accessing OpenRAG

In this task, you will log into IBM Cloud and navigate to the OpenRAG user interface.

In some of the following instructions, you will need to name resources based on your IBM Cloud username, which has the form of an email address. When asked to provide your username (`${USERNAME}`), use the email address. When asked for your user ID (`${USER_ID}`), use the portion of your username preceding the `@` symbol.

Example

IBM Cloud username: `student0@6aa075e639fdcf8a1ce36100.techzone.com`

```
${USER_ID}  = student0
${USERNAME} = student0@6aa075e639fdcf8a1ce36100.techzone.com
```

3.1. Logging in to IBM Cloud

1. Navigate to IBM Cloud using the following URL:

   https://cloud.ibm.com/authorize/2376bb61c1464da3b1b5e647becbe277/LBRrZD8TRZ

2. Log in using the provided IBM Cloud credentials.

3.2. Creating an IBM Cloud API key

1. Open the **Manage** menu and click **Access (IAM)** (see Figure 2).
2. Click **API keys** on the left.
3. Click **Create.**
4. Use your user ID as the name of the new IBM Cloud API key.
5. Copy the IBM Cloud API key and paste it, for example, into the **Editor** application.

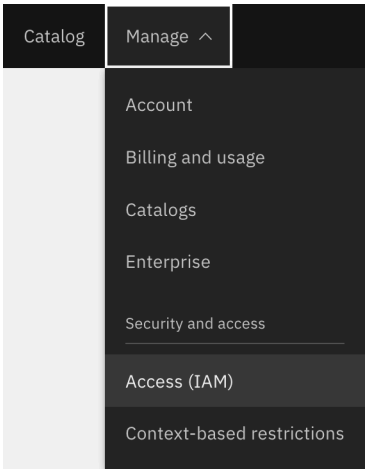


Figure 2: Access (IAM) menu item

3.3. Navigating to the watsonx.data web console

1. Click the **Resource list** button (☰) in the left menu.
2. Expand the **Databases** section.
3. Click on the watsonx.data instance.
4. Click **Open web console.**
5. Close the welcome screen.

3.4. Navigating to the OpenRAG console

Below the welcome message, locate the quick actions carousel and click the **OpenRAG** tile to launch the OpenRAG console (see Figure 3).

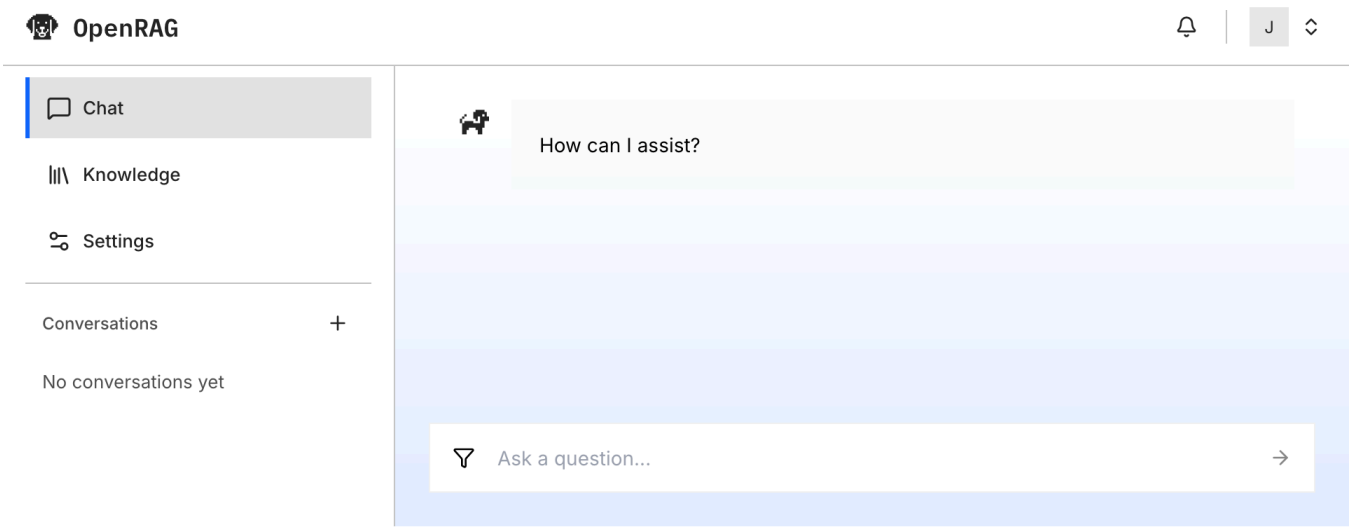


Figure 3: OpenRAG web console: Chat view

The left sidebar contains the navigation menu and the conversation history. The main content area displays the active view. There are three menu items:

- **Chat:** Chat feature to interact with the knowledge through natural language queries.
- **Knowledge:** OpenSearch knowledge base
- **Settings:** Management of cloud storage connectors, model providers, and common parameters for the Agent and Knowledge Ingestion flows.

You can start a new conversation by clicking the + button.

Finally, the button in the top navigation bar opens or closes a right sidebar with active and past ingestion tasks.

4. Task: Ingesting files

In this task, you will ingest the OpenRAG documentation into the knowledge base.

Instructions:

1. Click **Knowledge** in the left sidebar.
2. Click **Add knowledge** in the main content area.
3. Click **Folder** (see Figure 4).

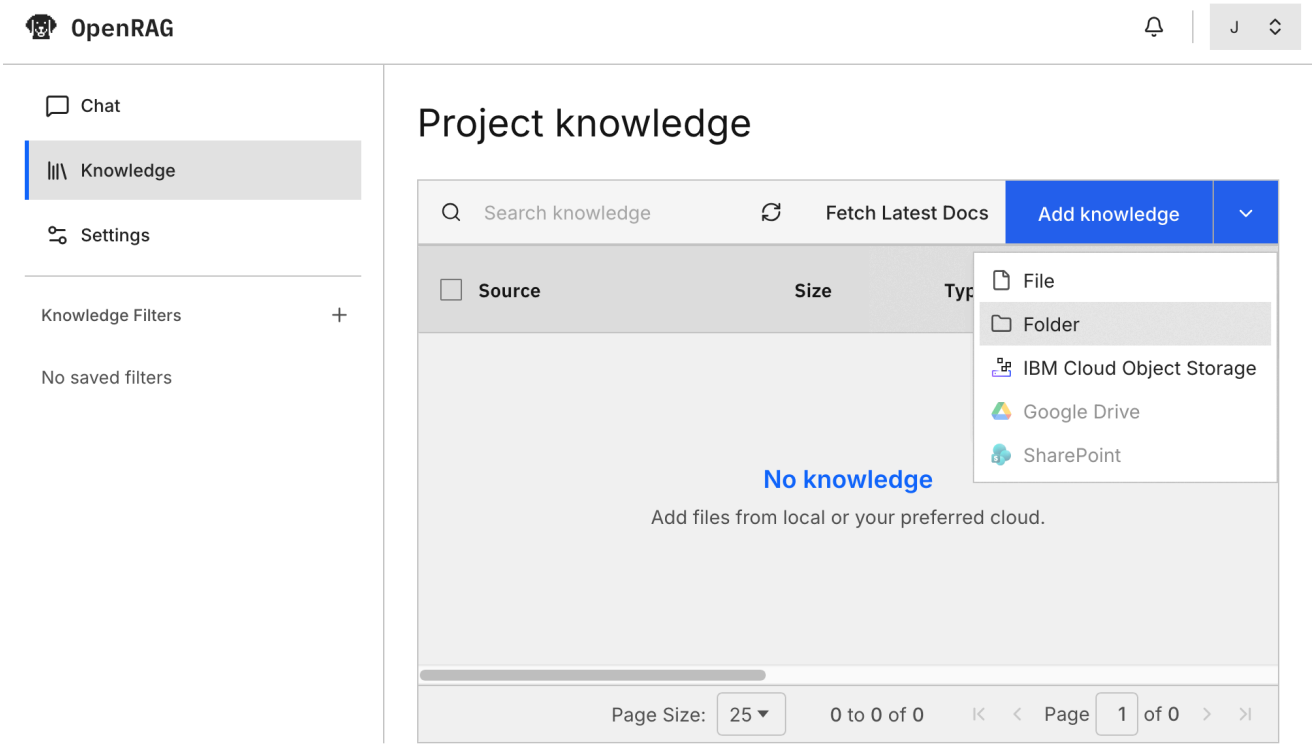


Figure 4: OpenRAG web console: Knowledge view

4. Navigate to the folder where you extracted the ZIP file that you downloaded in Section 2.1.
5. Select the folder named “OpenRAG documentation” and click “Upload”.
6. Click **Upload** on the confirmation dialog.

After confirming the upload, OpenRAG processes the uploaded HTML files in the background. Click the 🔔 button in the top navigation bar to monitor the progress of the ingestion process (see Figure 5).

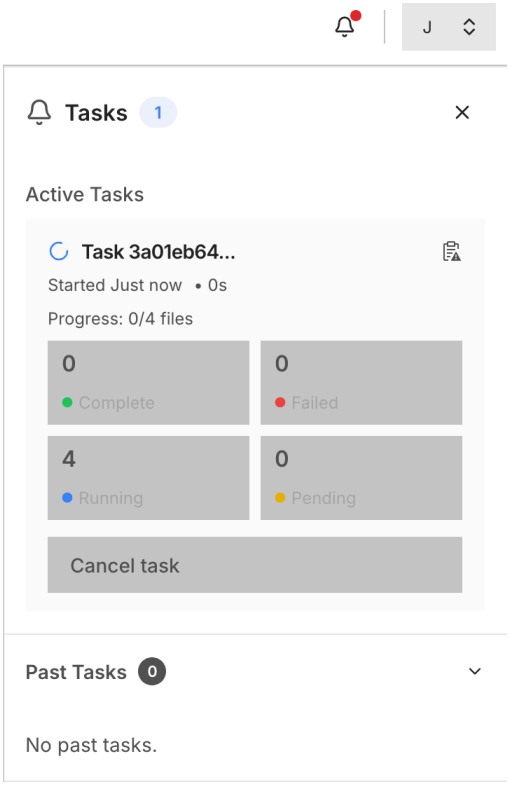


Figure 5: OpenRAG web console: Knowledge view

First, Docling parses and chunks the uploaded documents. Then, OpenSearch calls watsonx.ai to compute embeddings for the chunks and ingests them into an index.

Once ingestion is complete, you can see the list of ingested files below **Project knowledge** (see Figure 6).

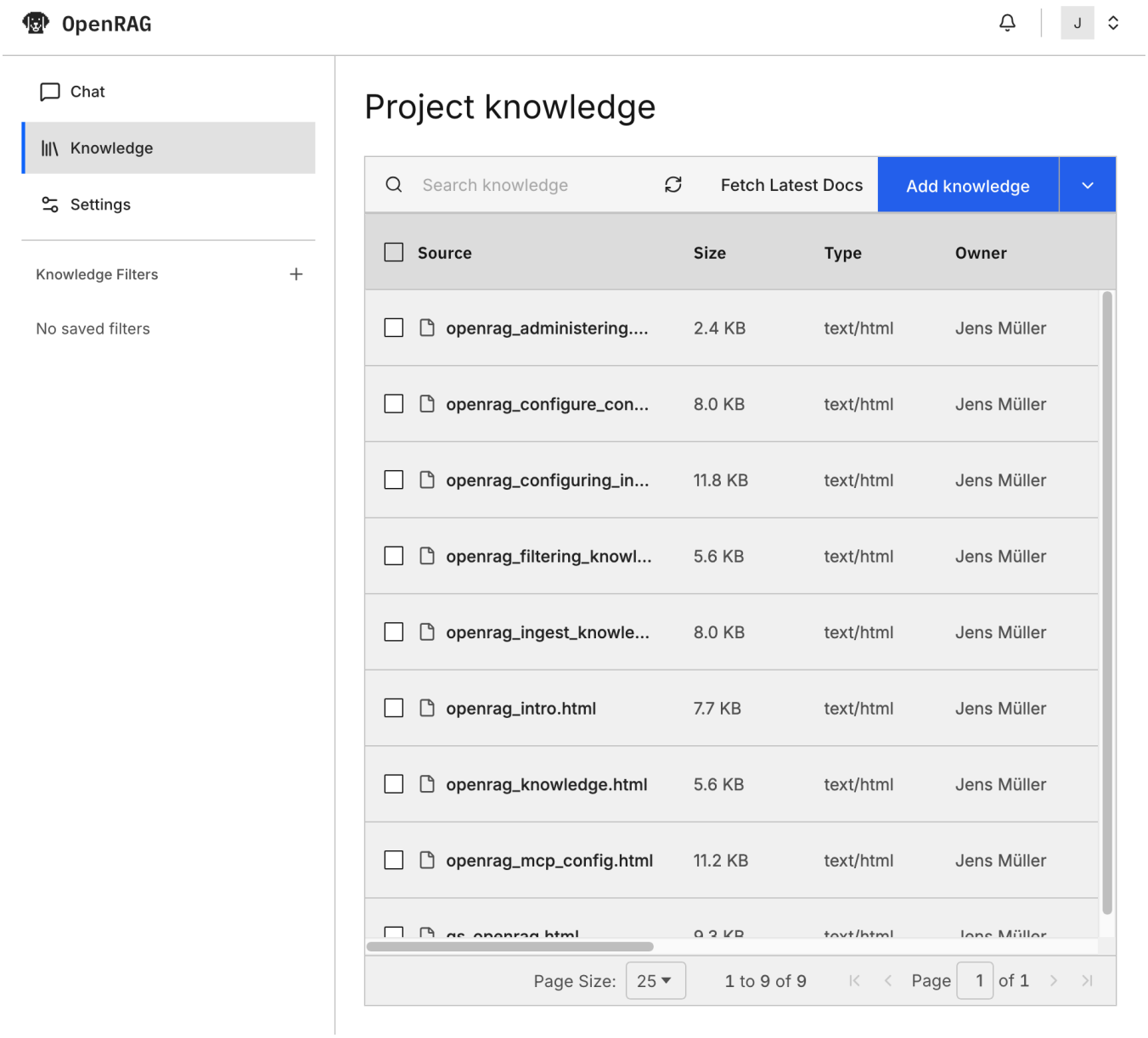


Figure 6: OpenRAG web console: Knowledge view listing ingested files

For each ingested file, the following information is shown:

Column name	Description
Source	Name of the ingested file
Size	Size of the ingested file in KiB

Column name	Description
Type	Mime type of the ingested file
Owner	Owner of the ingested file
Chunks	Number of chunks that the ingested file was split into
Avg score	Average similarity score of a document's matching chunks (shown only for results returned after a search query)
Embedding model	Name of the embedding model used for computing embedding vectors
Dimensions	Number of embedding model dimensions
Status	Document status (e.g., "Processing", "Active", etc.)

Clicking on a file opens a new view in the main content area, listing the chunks that the file was split into along with the total chunk count and their average length in characters (see Figure 7).

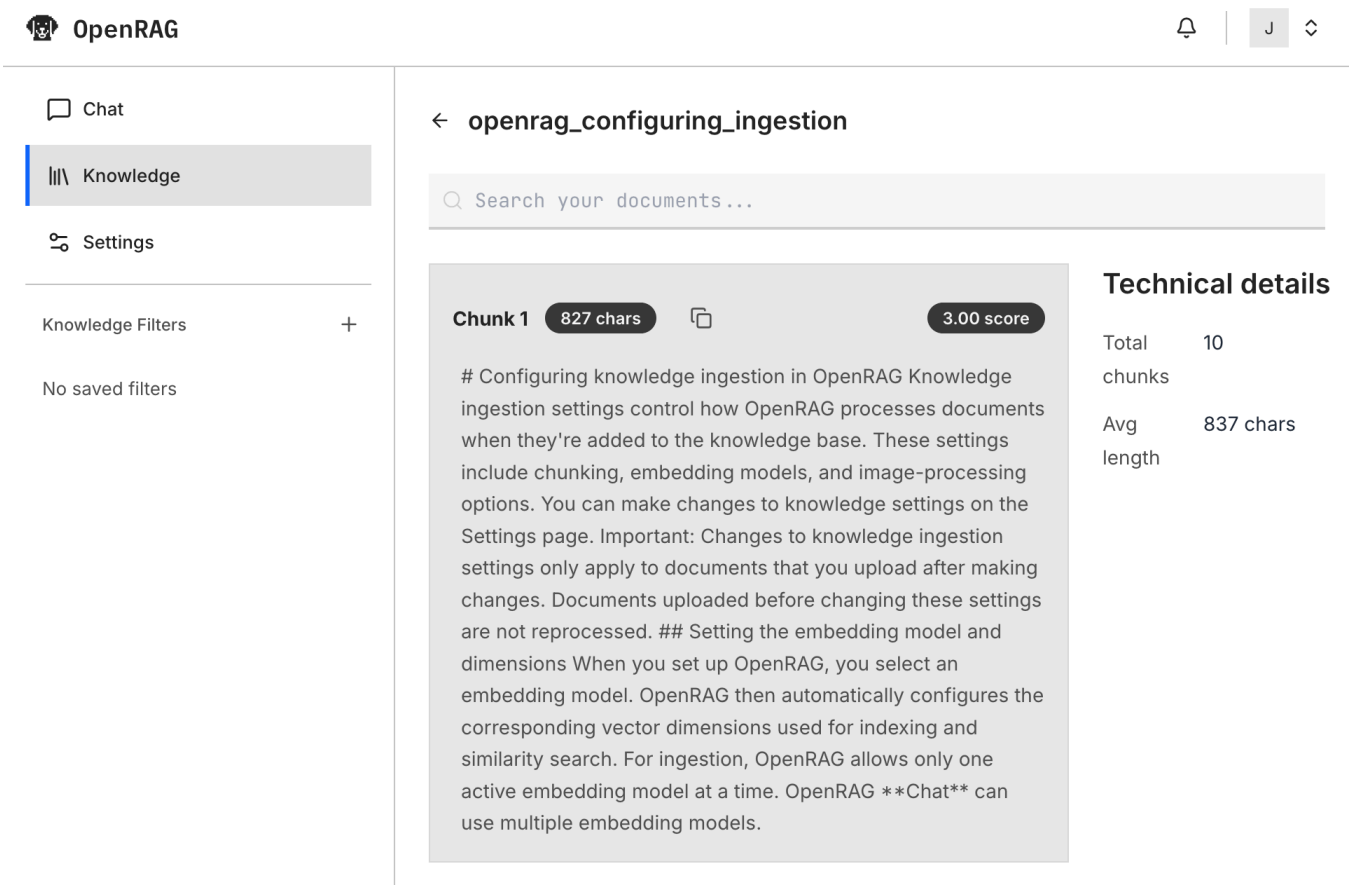


Figure 7: OpenRAG web console: Chunks view

Instructions:

1. Click on the **openrag_configuring_ingestion** document.

   This file is split into 10 chunks. Most chunks are similar in length. The last chunk is noticeably shorter — this is expected, as the final chunk simply contains whatever text remained at the end of the document.

   In general, chunk lengths can vary significantly across documents. OpenRAG splits documents on semantic boundaries such as paragraphs or sections rather than fixed character counts, so a short paragraph and a long section each become a single chunk. The uniformity you see here simply reflects that this particular document has a consistent structure.

2. Click ← or **Knowledge** in the left sidebar to return to the knowledge view.

5. Task: Querying documents

5.1. Searching documents

The **Knowledge** view contains a search bar below **Project knowledge.**

Instructions:

- Enter “MCP” into the search bar and press ⏎.

This triggers a query against the OpenSearch index and filters the document list to show only files whose chunks contain the search term. The **Avg score** column now shows the average similarity score of the matching chunks for each document. To remove the filter and return to the full document list, click the × button next to the search bar (see Figure 8).

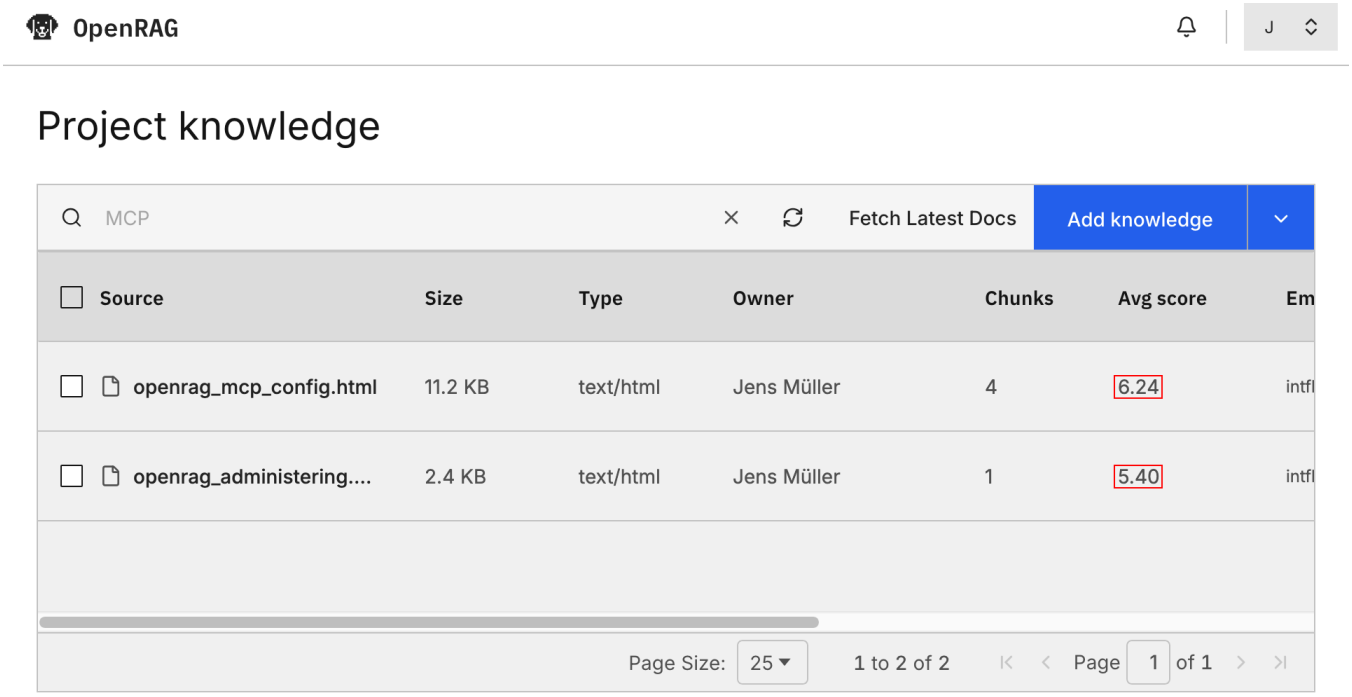


Figure 8: OpenRAG web console: Knowledge view with filtered documents

Note that this search operates at the document level — it shows which documents contain matching chunks, but does not show the chunks themselves. To inspect the matching chunks directly, click on one of the documents. This opens the chunks view, already filtered to show only the chunks containing your search term. To see all chunks of that document, click the × button next to the search bar in the chunks view to remove the filter.

If you want to open a document without any chunk pre-filtering, return to the **Knowledge** view, remove the document filter by clicking the × button next to its search bar, and then click the document.

5.2. Chatting with the agent

Now that you have explored the direct OpenSearch search, you will use the Chat feature to interact with the same knowledge base through an LLM-powered agent.

Instructions:

- Click **Chat** in the left sidebar.

Unlike the direct OpenSearch search you used earlier, the Chat feature uses an LLM-powered agent that retrieves relevant information from the knowledge base and generates a context-aware response.

For each question you ask, the agent decides how to use the `search_documents` tool — it may issue a single search, or it may decompose the question into multiple searches if it determines that different parts of the question require

retrieving information from different parts of the knowledge base. This decision is made by the LLM itself and depends on the model used.

You can inspect each individual tool call by clicking on the Function Call: `search_documents` entries that appear during the conversation — each entry shows exactly which query the agent issued to OpenSearch and what was retrieved (see Figure 9).

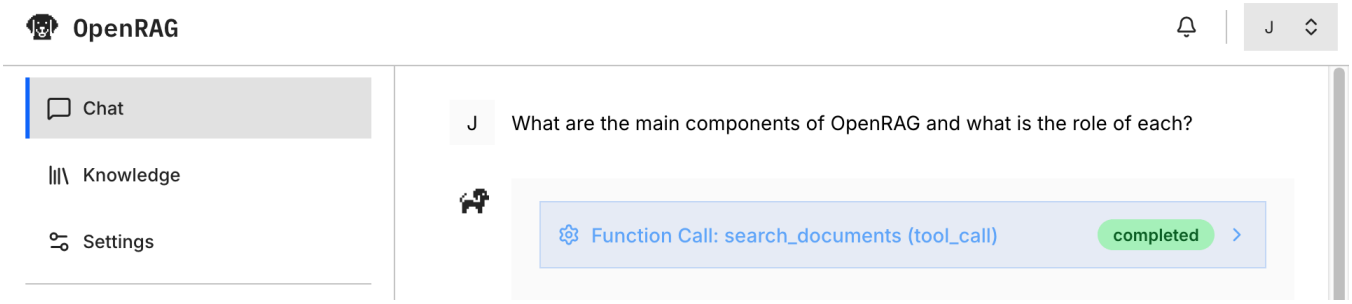


Figure 9: OpenRAG web console: Chat view showing function calls

Instructions:

1. For each of the following questions, enter it into the chat and inspect the agent's tool calls and response before proceeding to the next question:
   - What are the main components of OpenRAG and what is the role of each?
   - How does OpenRAG process a document during ingestion?
   - What happens if the embedding model used during ingestion is different from the one used during querying?
2. Click the + button to start a new conversation.
3. Ask the agent: "What is the weather like in Atlanta today?"

   Notice that the agent does not call the `search_documents` tool for this question — it has no relevant documents in the knowledge base to retrieve. The agent answers using its general knowledge instead, or acknowledges that it does not have access to real-time weather data.

5.3. Ingesting web content during a chat session

The Chat feature also supports ingesting web content directly during a conversation — the agent can fetch a URL you provide and use its content to answer follow-up questions.

Instructions:

1. Start a new conversation by clicking the + button.
2. Ask the agent: "Ingest https://www.ibm.com/events/techxchange"
3. Ask the agent: "When does IBM TechXchange 2026 take place?"

   The agent should respond that IBM TechXchange 2026 is scheduled for October 26 through October 29, 2026 in Atlanta, Georgia.

6. Task: Filters and querying filtered documents

In this task, you will create a connection to IBM Cloud Object Storage. Then, you will ingest the IBM earnings press releases for the first quarter from several years into the knowledge base from a bucket and query the three most recent quarters using knowledge filters.

6.1. Creating a connection to IBM Cloud Object Storage

Instructions:

1. Click **Settings** in the left sidebar.

   The **IBM Cloud Object Storage** tile on the **Connectors** tab should look like the left tile in Figure 10.

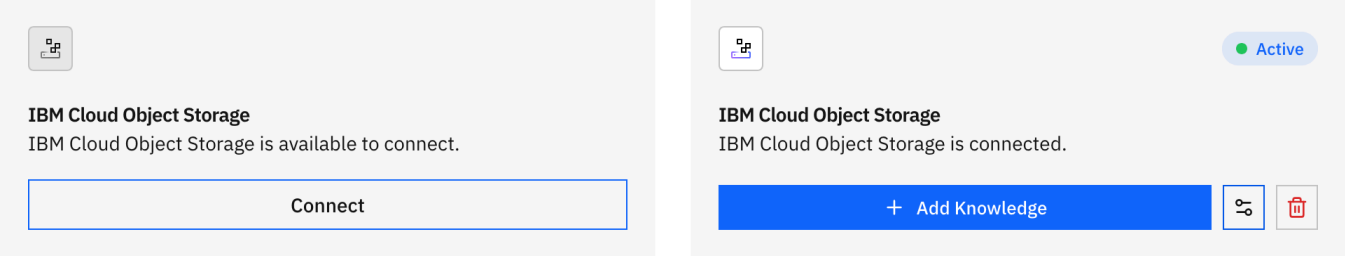


Figure 10: OpenRAG web console: IBM Cloud Object Storage tile before and after setup

2. Click **Connect** on the **IBM Cloud Object Storage** tile.
3. Enter the following values (see Figure 11):

Filter name	Selected sources
Access Key ID	6ef929e19c064203859473aa20868c04
Secret Access Key	716cc8af677765f28c87f1eb59064f430c7d1f36752496c3
Service Endpoint	https://s3.us-south.cloud-object-storage.appdomain.cloud

4. Click **Test Connection & List Buckets.** Afterwards, you should see a single bucket named “ibm-earnings-press-releases” below **Restrict Ingestion to Buckets.**
5. Click **Save.**

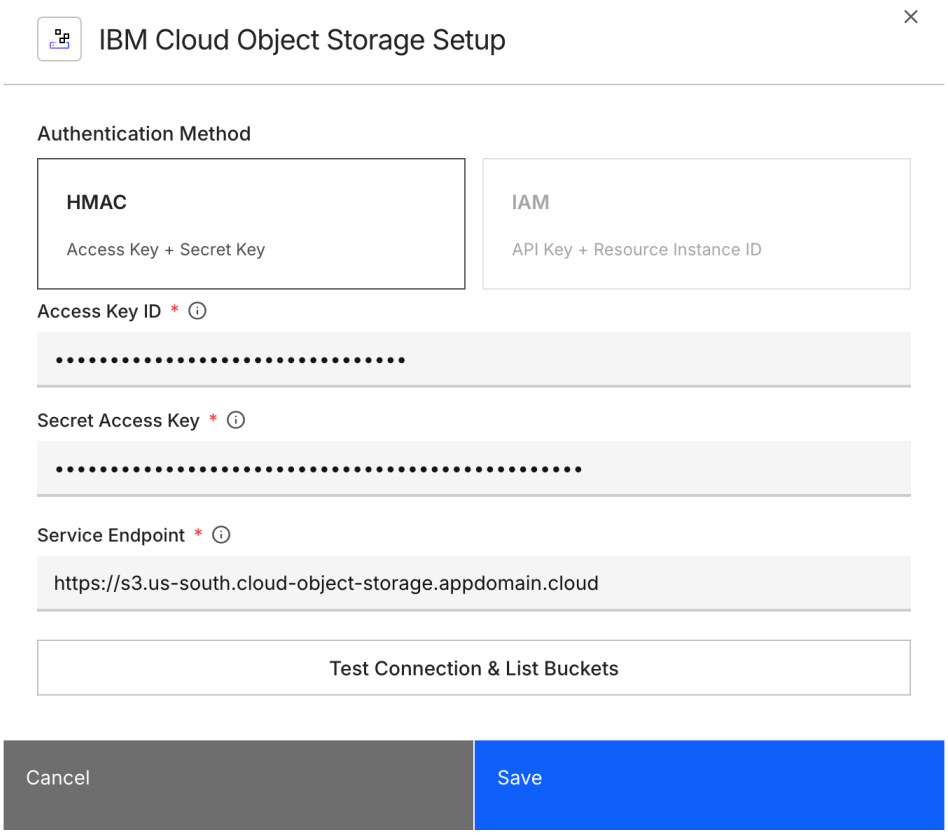


Figure 11: OpenRAG web console: IBM Cloud Object Storage setup

6.2. Ingesting files from IBM Cloud Object Storage

Now that the connection is configured, you can ingest the press releases directly from the bucket.

Instructions:

1. Click **Knowledge** in the left sidebar.
2. Click **Add knowledge** in the main content area.
3. Click **IBM Cloud Object Storage.**

   The main content area should now look like as shown in Figure 12.

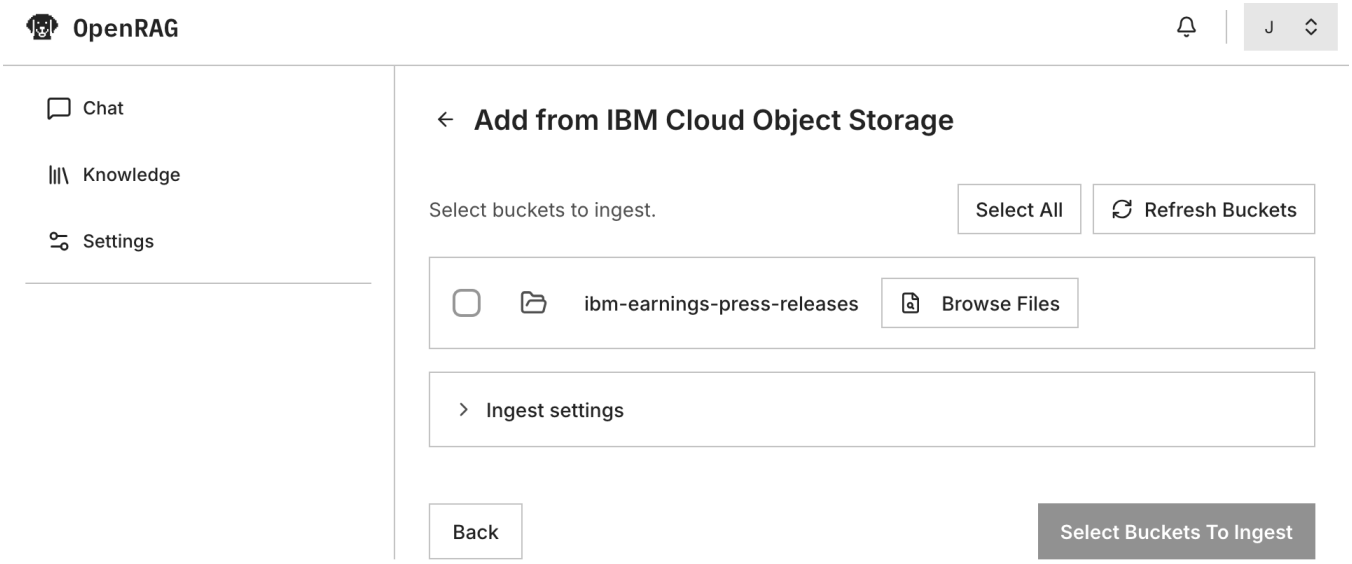


Figure 12: OpenRAG web console: Adding from IBM Cloud Object Storage

4. Select the bucket named “ibm-earnings-press-releases”.

> ⚠ **Warning**
>
> Do not enable **Make documents available to all users** under **Ingest settings** as shared documents are visible to all participants of the hands-on lab!

5. Click **Ingest 1 Container.**

Once ingestion is complete, you will first query the press releases without any filters to establish a baseline. Even when a question explicitly specifies a quarter and year, the agent does not always identify the correct document because it must choose from five structurally identical documents. However, questions that include a quarter and year generally provide a stronger signal and are more likely to return the intended document. When a question is more ambiguous and does not specify a year, there is no guarantee that the agent retrieves the document you intended, and the answer may refer to any of the five press releases.

This ambiguity is an inherent limitation of similarity search when applied to a corpus of highly similar documents. The embedding vectors of the five press releases are very close to each other, since the documents share the same structure, terminology, and topics. As a result, even questions that specify a quarter and year can occasionally retrieve the wrong document, while vaguer questions do not provide enough signal to reliably distinguish them.

In the following tasks, you will first observe this behavior, and then address it by creating knowledge filters — one for each quarter. A knowledge filter explicitly limits the documents the agent can access during a conversation, giving you precise, predictable control over which press release is used to answer a question, regardless of how the question is phrased.

Instructions:

1. Click **Chat** in the left sidebar.
2. Ask the agent: “What was IBM’s revenue in Q1/2024?”

   The agent should return the correct answer — it can identify the right document because the question explicitly specifies the quarter and year.

3. Ask the agent: “What was IBM’s revenue in the first quarter?”

   Notice that the answer is unpredictable — the agent may ask for clarification, return the revenue for one specific quarter, list the revenues for several quarters, or even return all five.

The unpredictability of the last answer illustrates the ambiguity introduced by having five structurally identical documents in the knowledge base. Now, you will use knowledge filters to address this.

6.3. Creating and using knowledge filters

Instructions:

1. Click **Knowledge** in the left sidebar.
2. Click the + button to create a new filter, which opens a right sidebar.
3. Create the following filters by setting **Filter name**, selecting files from the **Select sources...** drop-down list, and clicking **Create Filter**:

Filter name	Selected sources
Q1/2024	ibm-2024-q1-earnings-press-release.pdf
Q1/2025	ibm-2025-q1-earnings-press-release.pdf
Q1/2026	ibm-2026-q1-earnings-press-release.pdf

A knowledge filter is activated after creation or if you select it in the left sidebar (see Figure 13).

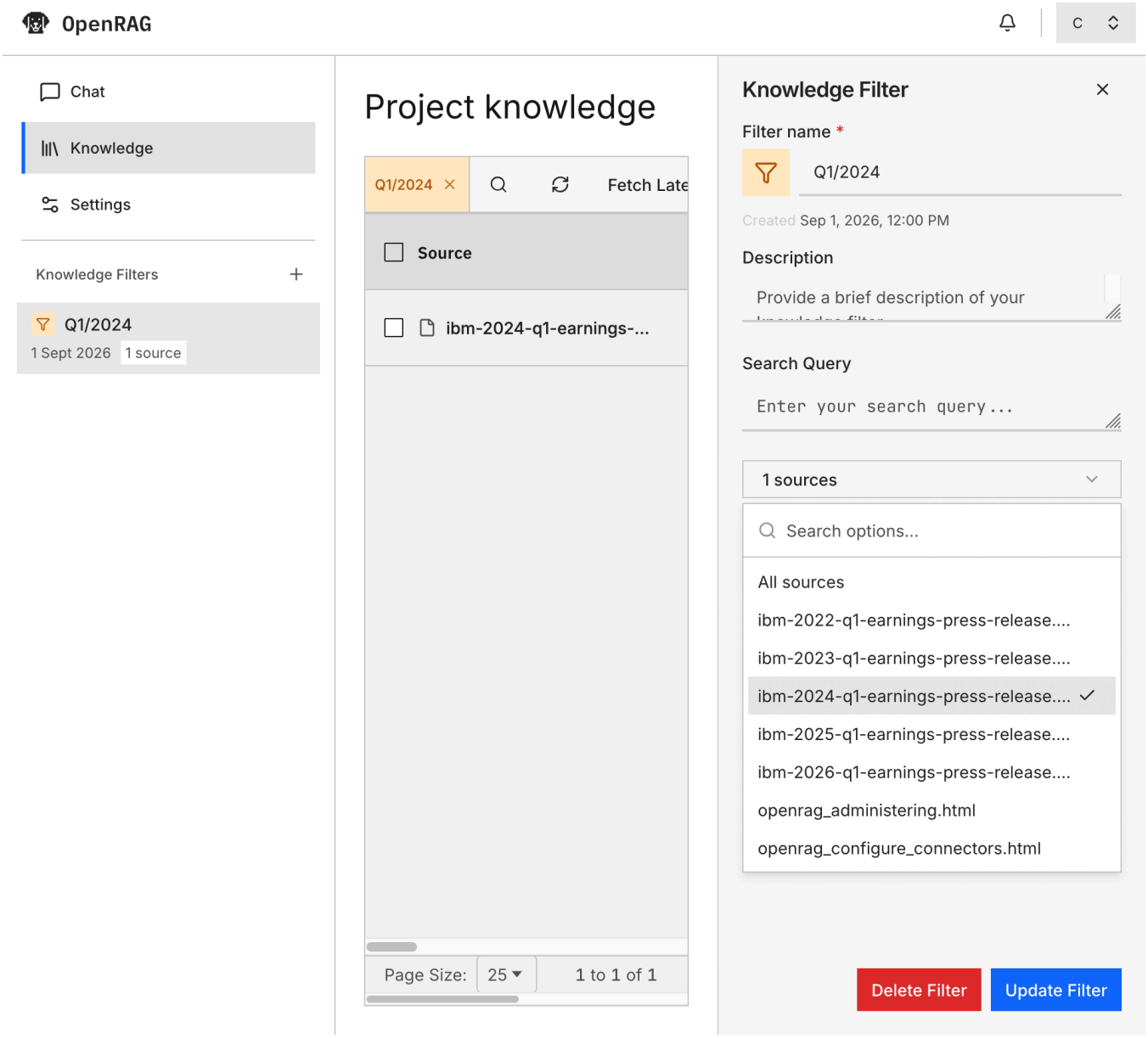


Figure 13: OpenRAG web console: Creating a knowledge filter

With the knowledge filters created, you will now repeat the ambiguous query from the previous task — this time with a filter active — to see how filters resolve the unpredictability you observed earlier.

Instructions:

1. Click **Chat** in the left sidebar.
2. Click the ▽ button and select one of the filters (see Figure 14).

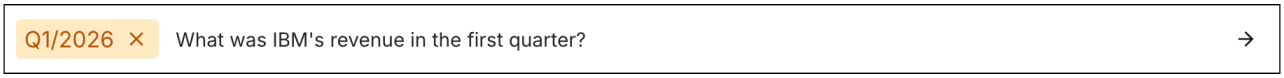


Figure 14: OpenRAG web console: Selecting a knowledge filter

2. Ask the agent: “What was IBM’s revenue in the first quarter?”

   Repeat this query three times, each time selecting a different filter, and verify that the agent returns the correct revenue for each quarter. Depending on the filter, the agent should respond with the following numbers:

Filter name	Revenue
Q1/2024	$14.5 billion
Q1/2025	$14.5 billion
Q1/2026	$15.9 billion

   Note that Q1/2024 and Q1/2025 correctly share the same revenue figure of $14.5 billion.

With a filter active, the agent can still reason over the full content of the filtered document. Try a more detailed question to see this in action.

Instructions:

1. Click the ▽ button and select the “Q1/2026” filter.
2. Ask the agent: “What were the main drivers of IBM’s revenue growth in the first quarter?”

7. Task: Langflow flows for ingesting and querying

OpenRAG includes two built-in Langflow flows that power its core functionality: the **Knowledge Ingest** flow, which processes and stores documents in the OpenSearch knowledge base, and the **Agent** flow, which handles chat queries by retrieving relevant chunks and generating responses. In this task, you will review both flows in the embedded Langflow visual editor to understand how OpenRAG works under the hood.

⚠ Warning

Do not modify the language models, the agent instructions, the Langflow flows (i.e., keep the flows locked), and all other settings as they are shared by all participants of the hands-on lab!

7.1. OpenSearch Ingestion Flow

Instructions:

1. Click **Settings** in the left sidebar.
2. Select the **Ingestion** tab.
3. In the **Knowledge Ingest** section, click **Edit in Langflow.**

4. Click **Proceed.**
5. Review the **OpenSearch Ingestion Flow** flow (see Figure 15):

Component	Explanation
Docling Serve	The entry point. Accepts a file upload and sends it to a remote Docling instance for document conversion. Docling handles PDFs, Word documents, images, and more.
Export DoclingDocument	Converts the Docling output into Markdown text, preserving document structure (headings, tables, lists).
Data Operations (×2)	Two sequential steps that attach fixed metadata to every document record — `file_size` and `mimetype` — so this information is stored alongside each chunk in OpenSearch and available for filtering.
Split Text	Splits the Markdown text into overlapping chunks of 1,000 characters with a 200-character overlap, so content spanning a chunk boundary is not lost.
Embedding Model	Converts each text chunk into a vector embedding.
OpenSearch	Stores the chunks and their embeddings in the OpenSearch index.

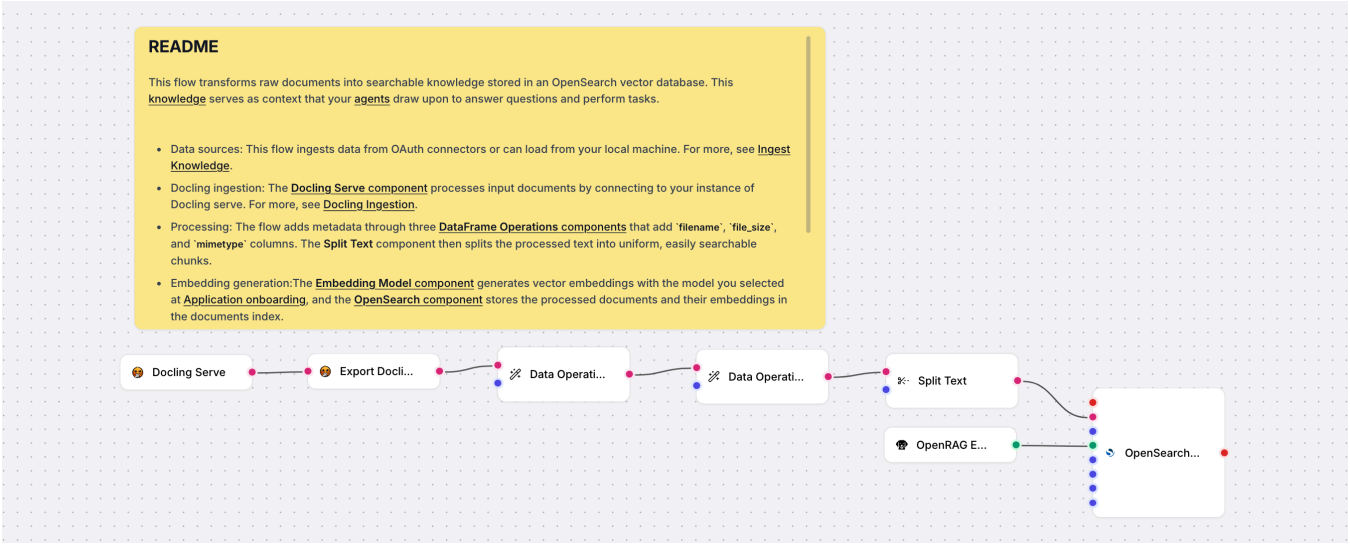


Figure 15: Langflow UI: OpenSearch Ingestion Flow

6. Close the Langflow browser tab and click **Leave page.**

7.2. OpenRAG OpenSearch Agent Flow

Now that you have reviewed the ingestion flow, switch to the Agent flow to see how OpenRAG handles chat queries.

Instructions:

1. Select the **Agent** tab.
2. In the **Agent** section, click **Edit in Langflow.**
3. Click **Proceed.**
4. Review the **OpenRAG OpenSearch Agent Flow** flow (see Figure 16):

Component	Explanation
Chat Input / Chat Output	The entry and exit points of the flow. Chat Input captures the user's message; Chat Output displays the agent's final response.
Text Input	A static knowledge filter — an optional text field that scopes what the agent should search for in OpenSearch. It feeds into both the Prompt Template and the OpenSearch component.
Prompt Template	Combines the knowledge filter text and the user's message into a single prompt that is passed to the Agent as its input.
Language Model	The LLM the agent uses for reasoning and generating responses (currently `openai/gpt-oss-120b` on watsonx.ai).
Agent	The core of the flow. It receives the prompt and decides which tool to call. Its Agent Instructions (system prompt) define its behavior and explain the available tools. It streams its response directly to Chat Output.
Tools wired into the Agent	• OpenSearch: Searches the indexed knowledge base. Uses the Embedding Model to embed the query before retrieval. • MCP Tools: Fetches and ingests web URLs when the user asks about external web content. • Calculator: Evaluates mathematical expressions.
Embedding Model	Converts the user's query into a vector for semantic search. Connected only to the OpenSearch tool, not to the LLM.

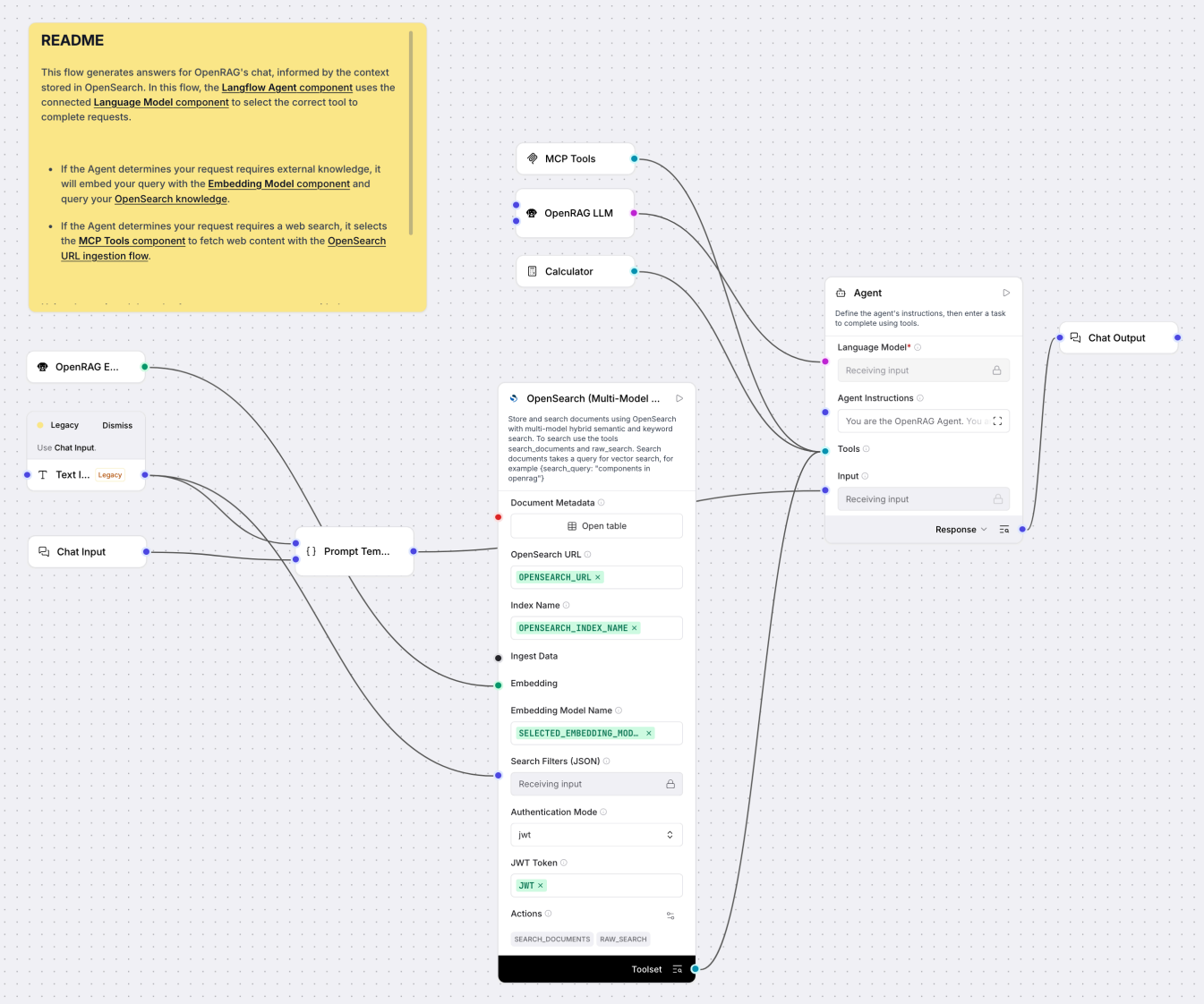


Figure 16: Langflow UI: OpenRAG OpenSearch Agent Flow

4. Close the Langflow browser tab and click **Leave page.**

7.3. Agent Instructions

Finally, review the agent instructions that guide the LLM's behavior during a chat session.

Instructions:

1. In the **Agent** section, review the **Agent Instructions.**

The default instructions define which tools the agent has access to, when to use each tool, and how to construct answers — including citation rules, retrieval decision rules, and special handling for file uploads versus web URLs.

Since the agent instructions are plain text, they can be extended to implement custom behaviors without modifying the Langflow flow itself. For example, you could add PII detection rules to prevent the agent from including sensitive information in its responses. The following addition to the instructions would instruct the agent to filter out e-mail addresses from its answers:

```
8. Before returning any response, scan the output for e-mail addresses (e.g., matching the pattern
user@domain.com) and replace any found with [REDACTED EMAIL].
```

This is just one example — the same approach can be used to enforce tone, restrict topics, or require specific response formats.

8. Task: Connecting watsonx Orchestrate with OpenRAG via MCP

8.1. Navigating to watsonx Orchestrate

Instructions:

1. Switch to the IBM Cloud tab (see Section 3.1).
2. Click the **Resource list** button (☰) in the left menu.
3. Expand the **AI / Machine Learning** section.
4. Click on the watsonx Orchestrate instance.
5. Click **Launch watsonx Orchestrate.**

8.2. Creating a connection to the OpenRAG MCP server

Instructions:

1. Click the main menu button (☰) in the upper left corner.
2. Under **GOVERN & OPERATE,** click **Connections.**
3. Click **Add connection.**
4. Set **Connection ID (Required)** as follows:

   ```
   connection-${USER_ID}
   ```

5. Click **Save and continue.**
6. Click **Continue.**
7. Select **Key Value Pair** from the **Authentication type** drop-down list.
8. Click **Add key value pair**.

9. Set the following key value pairs:

Key	Value
x-api-key	⇧ see Section 3.2
x-username	`ibmlhapikey_${USERNAME}`

Note that `${USERNAME}` is your IBM Cloud username, not your user ID (see Section 3).

Example

IBM Cloud username: `student0@6aa075e639fdcf8a1ce36100.techzone.com`

`x-username = ibmlhapikey_student0@6aa075e639fdcf8a1ce36100.techzone.com`

10. Click **Next.**
11. Click **Paste draft configuration.**
12. Click **Finish.**

8.3. Creating an agent

Instructions:

1. Click the main menu button (≡) in the upper left corner.
2. Click the **Home** button (⌂).
3. Click the **Create agent** button.
4. Click **Create from scratch.**
5. Set **Agent name** as follows:

   `agent-${USER_ID}`

6. Select the **Tools** tab.
7. Click **Add Tool**.
8. Click **MCP server.**
9. Click **Add MCP server.**
10. Click **Remote MCP server.**
11. Click **Next.**
12. Enter the following values:

Field	Value
Name	OpenRAG
MCP server URL	Use the URL of the OpenRAG console and replace `/chat` with `/mcp`
Select Connection (Optional)	connection-${USER_ID}

13. Click **Connect.**

    watsonx Orchestrate now connects to the OpenRAG MCP server and shows the available tools.

14. Select **OpenRAG:openrag_search** (see Figure 17).

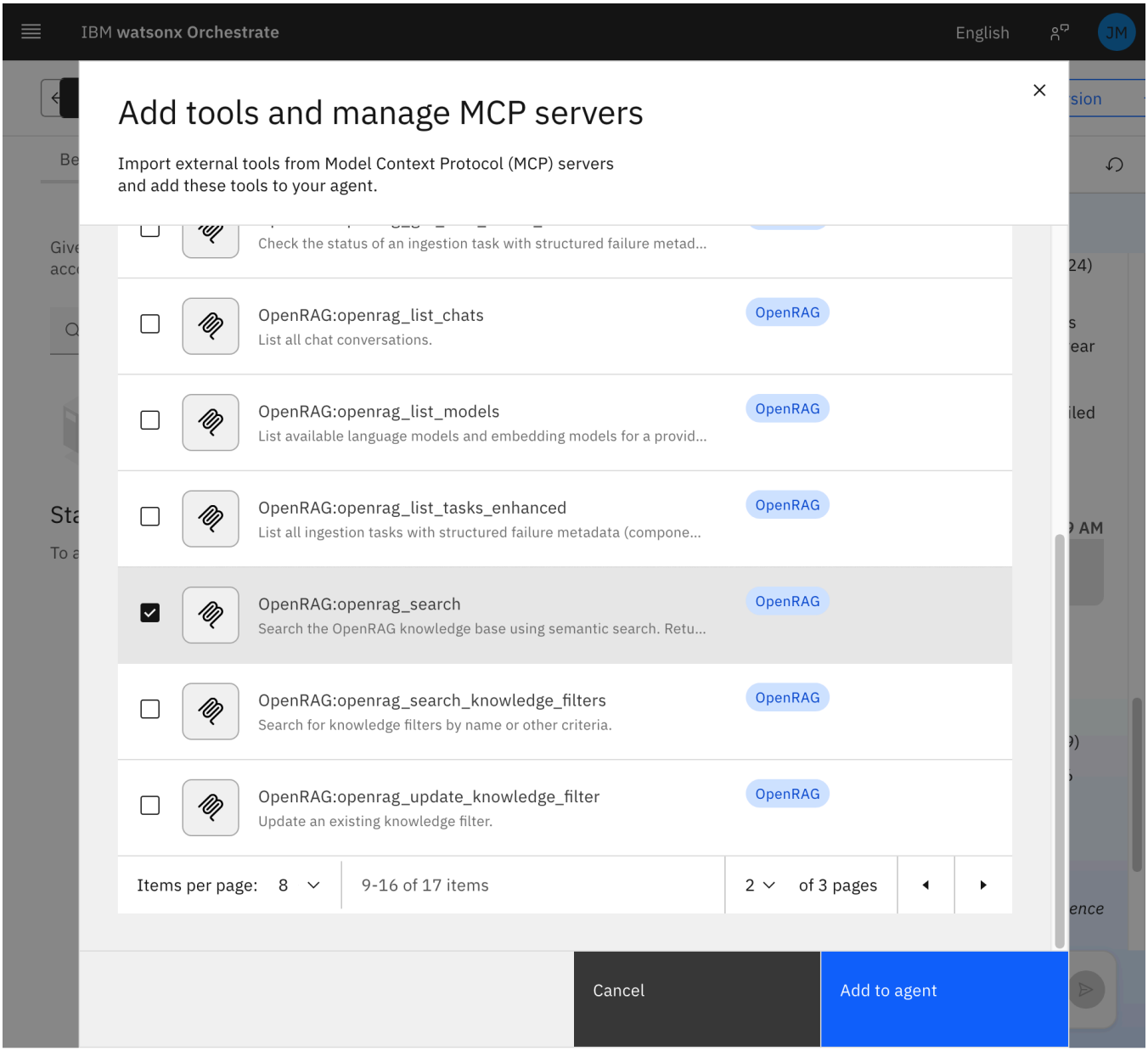


Figure 17: watsonx Orchestrate: Adding tools and managing MCP servers

15. Click **Add to agent.**

> **Tip**
>
> Note that the `openrag_chat` tool was not added here. Using it would place the OpenRAG Chat agent behind the watsonx Orchestrate agent, resulting in two nested LLM reasoning layers — one in Orchestrate and one in OpenRAG. This is redundant, adds latency, and makes the system harder to understand and debug. The `openrag_search` tool is the better choice as it keeps watsonx Orchestrate as the single reasoning layer while OpenRAG serves its intended role as a retrieval backend.

16. In the **Draft Preview** on the right side (see Figure 18), ask the agent:

    When does IBM TechXchange 2026 take place?

    This is the same question you asked in the OpenRAG Chat feature earlier — this time retrieved through watsonx Orchestrate via the OpenRAG MCP server.

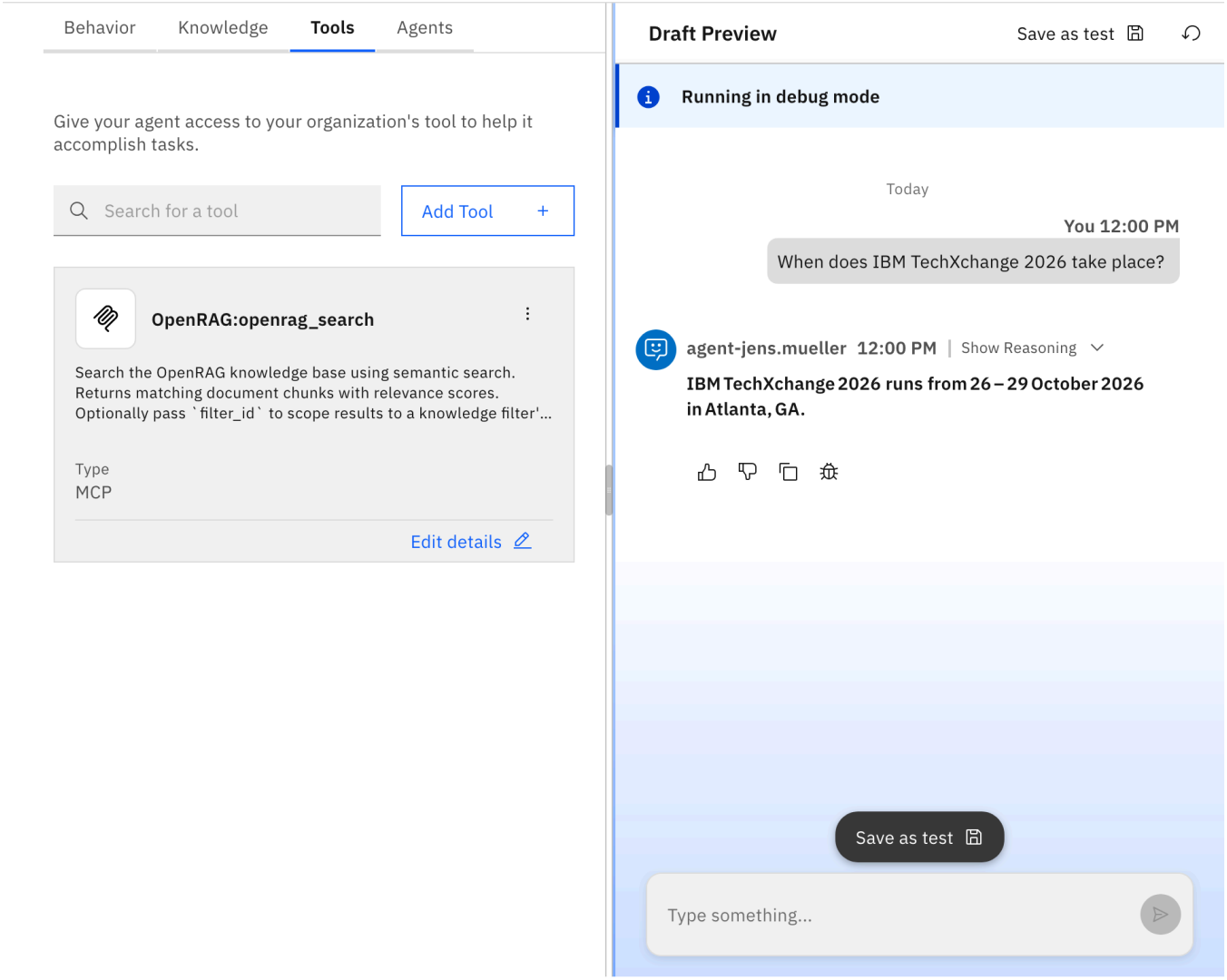


Figure 18: watsonx Orchestrate: Draft preview of the agent